OsirisCare

Autonomous HIPAA Compliance Infrastructure

NixOS + LLM Healing + Cryptographic Evidence Chain + Row-Level Security

222K+	121K+	23.5K	27	88
EVIDENCE BUNDLES	BITCOIN-ANCHORED PROOFS	HEALING EXECUTIONS	RLS-ENFORCED TABLES	EXECUTABLE RUNBOOKS

System Topology

CLIENT PRACTICE (ON-PREMISES)

NIXOS APPLIANCE (HP T640 / VM)

Compliance Agent + Daemon

Go daemon (v0.3.21): WinRM/SSH executor, fleet order processor, drift scanner, app discovery
Python agent: 1037+ tests, L1 deterministic engine, PHI scrubber, evidence signing
Key properties: Deterministic builds (Nix flakes), systemd hardened, nftables pull-only firewall, Ed25519 identity

→

MANAGED FLEET

Windows + Linux

Domain Controller (WinRM)
Workstations (per-host creds)
Linux servers (SSH)
25-50 devices per site

↓ HTTPS checkin every 60s — TLS 1.3 via Caddy — API key + Ed25519 ↓

CENTRAL COMMAND (HETZNER VPS, EU)

APPLICATION

FastAPI (Python 3.11) + React/Vite

Incident pipeline (L1/L2/L3/L4) • Fleet management • Checkin receiver • Evidence chain • OTS scheduler
4 portals: Admin, Partner, Client, Companion • OAuth + TOTP 2FA • CSRF double-submit • RBAC (4 roles)

DATA

PostgreSQL 16 + PgBouncer + Redis 7

27-table RLS (FORCE) • GUC tenant context
Transaction pooling • `statement_cache_size=0`
Rate limiting • Session cache • AOF persistence

EVIDENCE

MinIO WORM + OpenTimestamps

COMPLIANCE-mode Object Lock (90 days)
4 OTS calendar servers • Bitcoin anchoring
Ed25519 signatures • HMAC-SHA256 hash chain

↓

EXTERNAL INTEGRATIONS

AI

Anthropic Claude API

L2 incident planner • Runbook selection • PHI-scrubbed metadata only

TIMESTAMPING

Bitcoin via OpenTimestamps

SHA-256 anchoring • 4 calendar servers • Independently verifiable

NOTIFICATIONS

Email / Slack / PagerDuty / Teams

Multi-channel L3 escalation routing • Per-partner config • Site overrides

AUTH

Microsoft / Google OAuth

Partner portal SSO • PKCE flow • Session + TOTP 2FA

Four-Tier Healing Pipeline + Data Flywheel

TIER 1	TIER 2	TIER 3	TIER 4	FLYWHEEL
Deterministic	**LLM Planner**	**Partner Escalation**	**Central Command**	**L2 → L1**
95%	**5%**	**<1%**	**<1%**	↻
<100ms • $0 22,448 executions	2-5s • ~$0.001 1,126 executions	SLA-tracked ticket 670 tickets created	Admin queue Full audit trail	5+ successes @ 90%+ Auto-promote

L1: Pattern-Matched Rules

```
// l1_rules table (synced hourly) { "rule_id": "L1-SVC-DNS-001", "incident_pattern": { "incident_type": "service_stopped", "check_type": "dns_service" }, "runbook_id": "RB-AUTO-SERVICE_RESTART", "confidence": 0.95 }
```

Go daemon matches incident → executes runbook steps via WinRM/SSH → records evidence bundle. Zero external API calls.

L2: LLM Runbook Selection

```
# l2_planner.py async def analyze_incident(incident): # Load 88 runbooks (5-min TTL cache) runbooks = await load_all_runbooks() # Claude selects best runbook plan = await claude.complete( system=PLANNER_PROMPT, messages= [incident_context] ) if plan.confidence >= 0.6: return create_healing_order(plan) else: return escalate_to_l3()
```

Constrained to pre-approved runbooks. Cannot generate arbitrary commands. Self-reducing via flywheel.

L3/L4: Escalation Engine

```
// Escalation routing site → partner_id → notification_settings // Channels (per-partner config) email_recipients slack_webhook_url pagerduty_routing_key teams_webhook_url webhook_url + webhook_secret // SLA tracking sla_definitions.response_time_minutes escalation_tickets.sla_breached
```

Recurrence detection: same incident_type + site_id → recurrence_count++. L4 = partner escalates to central admin.

Cryptographic Evidence Chain

1 **Drift Scan** — WinRM/SSH compliance checks
→
2 **PHI Scrub** — 12 regex patterns on-appliance
→
3 **Ed25519 Sign** — Appliance private key
→
4 **Hash Chain** — prev_hash = H(bundle[n-1])
→
5 **WORM Store** — MinIO Object Lock 90d
→
6 **BTC Anchor** — OpenTimestamps → blockchain

EVIDENCE BUNDLE STRUCTURE

```
{ "bundle_id": "eb-2026-0311-a8f2c...", "site_id": "physical-appliance-pilot-1aea78",
"appliance_id": "ap-00:1a:2b:...", "created_at": "2026-03-11T14:42:16Z", "checks": [
{"type": "patching", "status": "pass", "details": ...}, {"type": "encryption", "status":
"fail", "details": ...} ], "prev_hash": "sha256:9f2a1b...", "signature":
"ed25519:a4c8e7...", "ots_proof": "base64:AQLF..." }
```

OPENTIMESTAMPS FLOW

```
# 1. Submit hash to 4 calendar servers calendars = [a.pool, b.pool, alice.btc, bob.btc]
pending = ots.stamp(sha256(bundle)) # 2. Background scheduler upgrades pending → anchored #
(waits for Bitcoin block confirmation) proof = ots.upgrade(pending) # ~1-2 hours # 3.
Anyone can verify independently ots.verify(proof, sha256(bundle)) # "anchored in Bitcoin
block 891,234"
```

PRODUCTION METRICS

222,599
TOTAL BUNDLES
Signed & hash-chained

121,457
BTC ANCHORED
Independently verifiable

16
PENDING UPGRADE
Awaiting block confirmation

0
CHAIN BREAKS
HMAC verification: 100%

WHY THIS MATTERS

✓ Auditor Independence

Auditors verify compliance state using Bitcoin proofs — no trust in OsirisCare infrastructure required. The proof is **mathematically independent** of the platform.

🔗 Append-Only Integrity

Hash chain means modifying any bundle invalidates all subsequent bundles. WORM storage prevents deletion. Combined: **tamper-evident by construction**.

🛡 Savepoint Isolation

Each proof upgrade runs in a PostgreSQL SAVEPOINT. One failed proof doesn't poison the batch transaction — handles PgBouncer prepared statement edge cases.

Multi-Tenant Isolation: Row-Level Security

RLS ARCHITECTURE

```
-- Every tenant table has two policies: CREATE POLICY admin_bypass ON incidents FOR ALL USING (
current_setting('app.is_admin', true)::boolean = true ); CREATE POLICY tenant_isolation ON incidents FOR ALL
USING ( site_id = current_setting('app.current_tenant', true) ); -- FORCE ensures even table owner obeys RLS
ALTER TABLE incidents FORCE ROW LEVEL SECURITY;
```

CONNECTION MIDDLEWARE

```
# tenant_middleware.py async with tenant_connection(pool, site_id="site-abc"): # SET LOCAL app.is_admin =
'false' # SET LOCAL app.current_tenant = 'site-abc' rows = await conn.fetch("SELECT * FROM incidents") #
Returns ONLY site-abc rows async with admin_connection(pool): # DB default: app.is_admin = 'true' rows = await
conn.fetch("SELECT * FROM incidents") # Returns ALL rows (admin bypass)
```

VERIFIED ISOLATION (PRODUCTION)

```
-- As mcp_app (non-superuser): SET LOCAL app.is_admin = 'false'; SET LOCAL app.current_tenant = 'pilot-
1aea78'; SELECT count(*) FROM compliance_bundles; -- 131,491 (tenant only) SELECT count(*) FROM
compliance_bundles WHERE site_id = 'test-lab-b3c40c'; -- 0 (cross-tenant blocked) -- Admin mode: SELECT
count(*) FROM compliance_bundles; -- 222,599 (all tenants)
```

27 RLS-PROTECTED TABLES

Core Data

incidents • compliance_bundles • l1_rules • l2_decisions • aggregated_pattern_stats • execution_telemetry

Fleet & Devices

appliances • orders • fleet_orders • discovered_devices • device_compliance_details • site_credentials

Evidence & Compliance

evidence_bundles • ots_proofs • compliance_scores • site_drift_config • protection_profiles • app_baselines

Escalation & Notifications

escalation_tickets • partner_notification_settings • site_notification_overrides • learning_promotion_candidates

PGBOUNCER INTEGRATION

Transaction Pooling Mode

`SET LOCAL` scoped to transaction — works with PgBouncer. Connection released back to pool after `COMMIT`. No tenant context leaks between requests.

Key config: `statement_cache_size=0` on asyncpg (prepared statements incompatible with transaction pooling).

AUTO-POPULATE TRIGGERS

6 tables auto-populate `site_id` from foreign keys on INSERT:

```
incidents (via appliance_id → appliances.site_id)
l2_decisions • orders • evidence_bundles
discovered_devices • device_compliance_details
```

Ensures RLS works even if application code omits site_id.

LLM Integration: Constrained, Auditable, Self-Reducing

L2 PLANNER ARCHITECTURE

```
# Input: PHI-scrubbed incident metadata { "incident_type": "service_stopped", "check_type": "dns_service", "severity": "high", "hostname": "DC01", "details": "DNS Server service not running" } # Output: runbook selection (NOT arbitrary commands) { "runbook_id": "RB-AUTO-SERVICE_RESTART", "confidence": 0.87, "reasoning": "DNS service critical for AD...", "escalate": false }
```

Guardrails

- **Selection only:** LLM picks from 88 pre-approved runbooks. Cannot generate commands.
- **Confidence gate:** ≥0.6 required for auto-execution. Below → L3.
- **PHI scrubbed:** 12-pattern regex runs on-appliance before data reaches API.
- **Runbook steps:** Each runbook has explicit WinRM/SSH commands. LLM doesn't write them.
- **Audit trail:** Every L2 decision logged in `l2_decisions` table with full reasoning.

Data Flywheel

When an L2 pattern succeeds 5+ times with 90%+ success rate, it's automatically promoted to an L1 deterministic rule. The system **continuously reduces its own AI dependency**.

Promoted rules: `L1-AUTO-{INCIDENT_TYPE}` — tracked in `aggregated_pattern_stats` + `learning_promotion_candidates`.

WHY THIS ARCHITECTURE

Selection Problem, Not Generation

Traditional LLM agents generate actions. Ours selects from a fixed catalog. This is a fundamentally different risk profile:

Arbitrary generation	Unbounded risk surface
Our approach: selection	88 possible outputs, all pre-audited
Worst case failure	Wrong runbook → no-op or safe failure

RESOLUTION FUNNEL (PRODUCTION)

88 Runbooks in Production

Each runbook: name, category, steps (ordered), per-step command (WinRM/SSH), expected output, rollback command. Loaded from DB with 5-min TTL cache. Steps execute sequentially with exit-on-failure.

279 LLM Decisions Logged

Every L2 invocation recorded in `l2_decisions`: runbook selected, confidence score, reasoning text, latency. 248 auto-eligible (≥0.6), 31 required human review. Full auditability of every AI decision.

Cost at Scale

1,126 L2 executions to date at ~$0.001 each = ~$1.13 total AI spend. As flywheel promotes patterns to L1, AI calls decrease over time. The system trends toward $0 marginal AI cost.

Deterministic Infrastructure: NixOS Appliance

WHY NIXOS

⚙ Reproducible by Construction

Every appliance is built from the same `flake.nix` + `flake.lock`. The Nix store is content-addressed: two identical inputs always produce the identical binary output. This is **cryptographic proof of configuration**.

🔒 systemd Hardening

- `ProtectSystem=strict` — read-only filesystem
- `PrivateTmp=true` — isolated temp directories
- nftables pull-only firewall (no inbound except established)
- SSH key-only authentication (no passwords)

📦 Golden Flake Deployment

1. `nix build .#appliance-iso` — build installer
2. Write to USB, boot target hardware
3. Auto-installs, auto-partitions (ESP + MSP-DATA + root)
4. First boot: phones home to Central Command
5. MAC lookup provisions identity + API key
6. Fleet orders deliver config updates remotely

GO DAEMON (V0.3.21)

```
// Core goroutines (all wrapped in safeGo recover) main() { go checkinLoop() // 60s heartbeat to Central Command go driftScanLoop() // Continuous compliance scanning go orderProcessorLoop() // Fleet order execution go appDiscoveryLoop() // Application baseline monitoring go windowsAutoDeployLoop() // WinRM software deployment } // Healing execution path incident → executeHealingOrder(order) { steps := loadRunbookSteps(order.RunbookID) for _, step := range steps { result := executeViaWinRM(target, step.Command) if result.ExitCode != 0 { break } } reportEvidenceBundle(results) }
```

FLEET ORDER SYSTEM

✍ Ed25519 Signed Orders

Every fleet order (config update, binary deploy, NixOS rebuild) is cryptographically signed. Go daemon verifies signature before execution. Nonce replay protection with 24h eviction window.

CREDENTIAL DELIVERY

Per-Host WinRM Credentials

Stored encrypted in DB. Delivered in checkin response. Daemon loads per-workstation creds: `LookupWinTarget(hostname)` returns host-specific credentials. DC creds used as fallback. Hostname → IP resolution handles AD name mismatches.

PHI Boundary & Security Controls

DATA CLASSIFICATION ZONES

✅ **System Metadata — Always Ingested**
Syslog, SSH events, package hashes, backup status, firewall rules, disk encryption state, service status. No patient identifiers.

⚠️ **Application Logs — PHI Scrubbed On-Appliance**
EHR audit logs, access events. 12-pattern regex strips SSN, MRN, names, DOB, credit cards, phone, email, insurance ID, drug names, ICD-10/CPT codes, lab values, diagnosis text. Matched → [REDACTED] + one-way hash.

🛑 **Clinical Data — Never Transmitted**
Patient records, diagnoses, prescriptions, lab results, treatment plans. Architecturally excluded at the appliance layer. Cannot reach Central Command.

KEY INSIGHT FOR AI SAFETY

The Claude API receives **incident metadata only**: check types, severity levels, hostnames, timestamps. Even if PHI scrubbing fails, the data flowing to the LLM is system operational context — not clinical data. This is defense-in-depth: architectural exclusion + active scrubbing + metadata-only API input.

SECURITY STACK

LAYER	IMPLEMENTATION	STATUS
Authentication	bcrypt-12, TOTP 2FA, OAuth PKCE, API keys	Live
Session Mgmt	HMAC-SHA256 tokens, 15-min idle, 24h absolute	Live
CSRF	Double-submit cookie, per-path exemptions	Live
IDOR Prevention	`require_site_access()` on all site endpoints	Live
Tenant Isolation	PostgreSQL RLS FORCE on 27 tables	Live
Transit Encryption	TLS 1.3 (Caddy + Let's Encrypt auto-renew)	Live
Audit Integrity	6 append-only tables with DB triggers	Live
Evidence Signing	Ed25519 + HMAC-SHA256 chain + OTS	Live
Rate Limiting	Redis-backed, 5 failures → 15-min lockout	Live
Fleet Order Auth	Ed25519 signature + nonce replay protection	Live

HIPAA §164.308-312 Control Mapping

REQUIREMENT	CITATION	TECHNICAL IMPLEMENTATION	EVIDENCE ARTIFACT	STATUS
Risk Analysis	§308(a)(1)(ii)(A)	10-asset risk matrix: threats, existing controls, residual risk scores	`RISK_ANALYSIS.md`	Implemented
Access Control	§312(a)(1)	bcrypt-12 + TOTP 2FA + RBAC (admin/operator/readonly/companion) + OAuth PKCE	`admin_audit_log`	Implemented
Automatic Logoff	§312(a)(2)(iii)	15-min session idle timeout, enforced server-side on every request	Session middleware	Implemented
Audit Controls	§312(b)	6 append-only audit tables + immutability triggers (BEFORE UPDATE/DELETE)	Migration 084 triggers	Implemented
Integrity Controls	§312(c)(1)	Ed25519 bundle signing + HMAC-SHA256 hash chain + Bitcoin OTS anchoring	`evidence_bundles` + `ots_proofs`	Implemented
Transmission Security	§312(e)(1)	TLS 1.3 via Caddy, auto-renewed Let's Encrypt, valid through May 2026	Caddy config + cert check	Implemented
Contingency Plan	§308(a)(7)	Continuous backup drift check + evidence WORM retention (90-day Object Lock)	Drift scan results	Implemented
Breach Notification	§164.400-414	8-step incident response runbook, 48-hour notification SLA, 4-factor risk assessment	`BREACH_RUNBOOK.md`	Implemented
BAA / Sub-Processors	§164.502(e)	9 sub-processors documented with data classification and BAA status	`BAA_SUBPROCESSORS.md`	Implemented
Information Access	§308(a)(4)	RLS tenant isolation (27 tables FORCE) + require_site_access IDOR guards	RLS policies + middleware	Implemented
Encryption at Rest	§312(a)(2)(iv)	MinIO WORM encrypted; VPS root volume encryption: planned	MinIO config	Partial

47
DRIFT CHECKS
Configurable per-site

6
AUDIT TABLES
Append-only + triggers

12
PHI PATTERNS
On-appliance scrubber

4
PORTALS
Admin, Partner, Client, Companion

OsirisCare

Compliance should be a property of the system, not a project.

23,577 autonomous healing executions. 222K evidence bundles. 121K Bitcoin-anchored proofs. Built by one engineer.

NixOS	Claude	Ed25519	Bitcoin	RLS
REPRODUCIBLE BUILDS	RUNBOOK SELECTION	EVIDENCE SIGNING	PROOF ANCHORING	TENANT ISOLATION

osiriscare.net